

K. Ganesh

Regno: 192525253

Course Code: CNA 1101

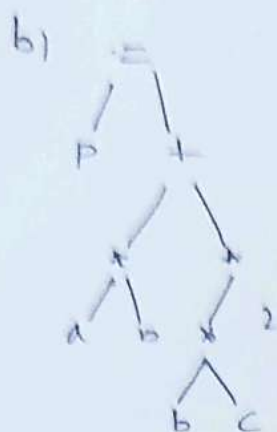
Course Name: COMPILER
DESIGN

Col - AT1

1. Trace the processing of the input expression $P = (a * b) + (b * c) * 2$ through all phase of a compiler
- Identify and list all tokens generated during lexical analysis
 - Construct the syntax tree during the parsing phase
 - Generate the intermediate code for the given expression
 - Apply suitable optimizations to the intermediate code.
 - Explain the significance of the final target code prod.

- a) Token generated during lexical analysis

Lexeme	Token
P	Identifier
=	Assignment Operator
(	Left parenthesis
a	Identifier
*	Multiplication Operator
b	Identifier
)	Right parenthesis
+	Addition Operator
(	Left parenthesis
b	Identifier
*	Multiplication Operator
c	Identifier
)	Right parenthesis
*	Multiplication operator
2	Integer Constant



c

$$\begin{aligned}
 t_1 &= a * b \\
 t_2 &= b * c \\
 t_3 &= t_2 * c \\
 t_4 &= t_1 + t_3 \\
 P &= t_4
 \end{aligned}$$

d

$$\begin{aligned}
 t_1 &= a * b \\
 t_2 &= b * c \\
 t_2 &= t_2 * c \\
 P &= t_1 + t_2
 \end{aligned}$$

- c. Machine-executable code generated by the compiler.
- Optimized for faster execution and efficient memory usage.
 - Can be directly executed by the processor.

2. a) $(a)^*$

- Language: Any number of a's.
- Strings: ϵ , a, aa, aaa, aaaa

b) $(b)abb$

- Language: Any number of b's followed by abb.
- Strings: abb, babbb, bbabbb, bbbabbb, bbbbabbb

c) $(ab)^*$

- Language: Any combination of a and b.
- Strings: ϵ , ab, aab, abb, aaabbb

d) $(ab)^*$

Language: Repetition of ab

- Strings: ϵ , ab, abab, ababab, abababab

e) $(aba) + (ab)^{**}$

- Strings: aba, abaaba, ϵ , aaabbb, ab

3. Tokens and Lexical Errors

i) Number of Tokens

Program:

```
Main()
{
    int a = 10;
    char b = "abc";
    int c = 30;
    char d = "xyz";
    int m = 40.5;
}
```

Total Tokens = 25

ii) Lexical Errors

$x = 1xabj$

• $1xab$ is an invalid numeric literal

• valid forms : $0xAB$ (hexadecimal) or a valid decimal number

4. Regular Expressions

1. Even number of 0's:

$(1^*01^*01^*)^*$

2. At least one 1:

0^*10^*

3. Odd number of both 0's and 1's:

$1^*(01^*01^*)^*01^*(10^*10^*)^*0^*$

4. starts with 1 and ends with 0:

$$1(0+1)^*0$$

5. No consecutive 1's:

$$(0+10)^*1?$$

5. Lexical Analyzer

input:

```
int var123 = 12345;
```

```
float var456 = 123.45;
```

```
if (var123 == var456) { return; }
```

a) Total Number of Tokens

18 Tokens

b) Tokens by Type

Keywords : int, float, if, return

Identifiers : var123, var456, var123, var456

Operators : =, =, =

Separators : ,, ,, (,), {, }, ;

Integer Literal : 12345

Floating Literal : 123.45

c) FSM State Transitions

Approximate transitions:

• Keywords : 17

• Identifiers : 24

• Integer Literal : 5

• Floating Literal : 6

• Operators & Separators : 10

Estimated Total FSM
State Transitions
= 62